

PYTHON

Dr. **Bülent Çobanoğlu**

cobanoglubulent[at]gmail.com

İçindekiler

Module 2:

- Standart Veri Tipleri
- Python Veri tipleri ve Tip Dönüşümleri
- Operatorler
- Karşılaştırma operatörleri
- Mantıksal operatörler
- True- False döndüren ifadeler
- any, all
- String formatlama (f-string)
- String fonksiyonlar
- Karar yapıları ve döngüler

Dünkü derste yeni öğrendiğiniz iki şeyi yazabilir misiniz?

1

2

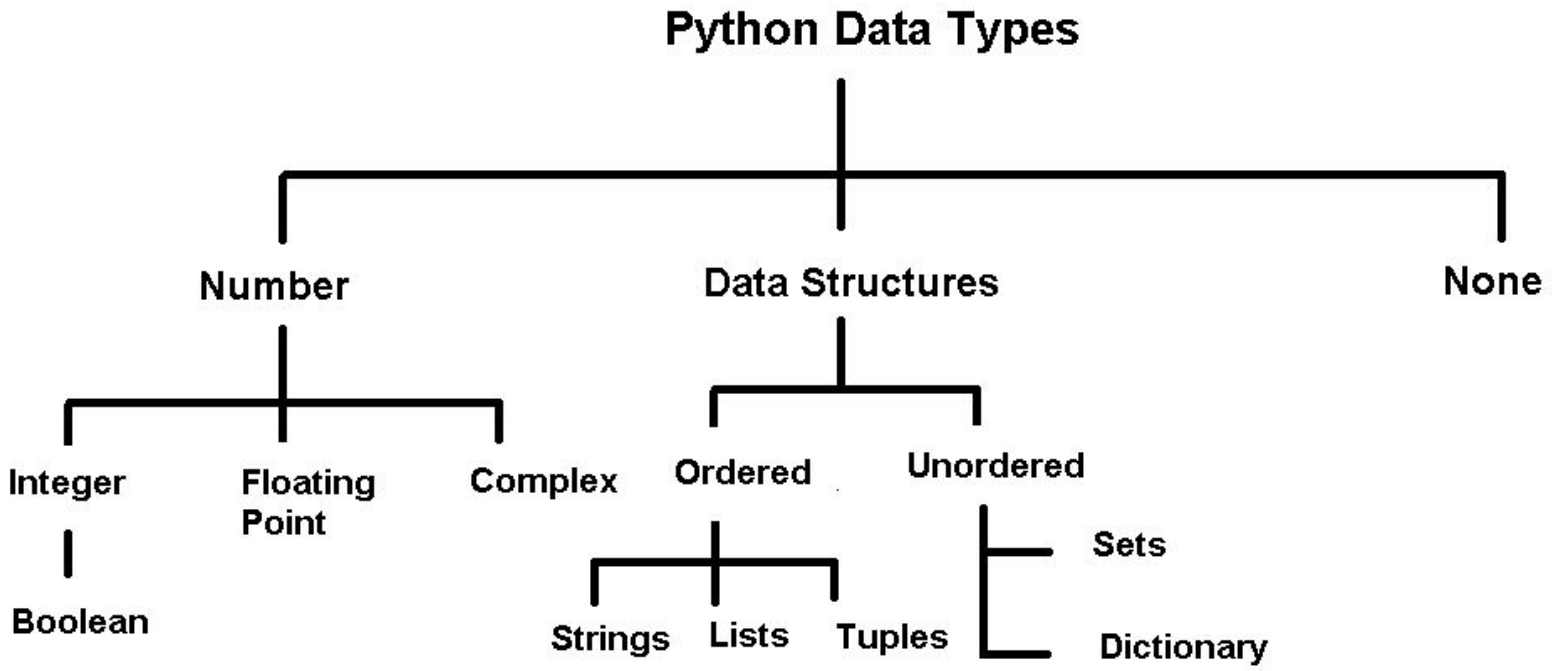


Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Standart Veri Tipleri (Data Types)

Python Veri Tipleri



Sayısal Veri Tipleri

- int
- float
- complex
- bool

Tam sayılar (integers)

- `Tamsayı veri tipi:` **`int`**
- Python dilinde, bir tamsayı değerinin ne kadar uzun olabileceğine dair **herhangi bir sınır yoktur.**

- Elbette bu sınır, her şeyde olduğu gibi sisteminizin sahip olduğu bellek miktarına bağlıdır.

Tam sayılar (integers)

- Python'da int veri tipinin maksimum değeri **sys.maxsize** sabiti ile bulunabilir.
- Bu sabit, 64 bit bir mimari üzerinde çalışan sistemler için 9223372036854775807 veya **2^63 - 1** değerini döndürür.
- Ancak 32 bit bir mimari üzerinde çalışan sistemler için 2147483647 veya **2^31 - 1** değerini döndürür.

Tam sayılar (integers)

Onlu (desimal) sayı sisteminden başka aşağıdaki sayı sistemleri de kullanılabilir

Ön Takı (Prefix)	Yorumlayıcı	Sayı Tabanı
0b (zero + lowercase letter 'b') **0B** (zero + uppercase letter 'B')	Binary(İkili)	2
0o (zero + lowercase letter 'o') **0O** (zero + uppercase letter 'O')	Octal(Sekizli)	8
0x (zero + lowercase letter 'x') **0X** (zero + uppercase letter 'X')	Hexadecimal(Onaltılı)	16

Tam sayılar (integers)

Onlu (desimal) sayı sisteminden başka aşağıdaki sayı sistemleri de kullanılabilir

```
>>> print(0o10)
8

>>> print(0x10)
16

>>> print(0b10)
2
```




```
>>> type(10)
<class 'int'>
>>> type(0o10)
<class 'int'>
>>> type(0x10)
<class 'int'>
```

SAYI SİSTEMLERİNİN DÖNÜŞÜMÜ

Python dilinde bir sayı sistemini diğer sayı sistemlerine (ikili, onlu, sekizli ve onlatılı) dönüştürüp ekranda göstermek için print ile birlikte ;

‘**0b’ (ikili), ‘0x’(onaltılı), ‘0o’ (sekizli)** çıkış formatı karakterleri veya ‘**bin(ikili)’, ‘hex(onaltılı)’, ‘oct(sekizli)’** built-in fonksiyonları kullanılabilir.

```
sayi = 10
print(f"{sayi:0b}")
print(f"{sayi:0x}")
print(f"{sayi:0o}")
```

✓ 0.0s

```
1010
a
12
```

Eşdeğeri

```
sayi = 10
print(bin(10))
print(hex(10))
print(oct(10))
```

✓ 0.0s

```
0b1010
0xa
0o12
```

SAYI SİSTEMLERİ

Buna göre onluk sistemde verilen 10 tam sayısı, aşağıdaki gibi dört farklı şekilde gösterilebilir;

```
sayi = 10
bin10 = 0b1010
oct10 = 0o12
hex10 = 0xA

print(sayi)
print(bin10)
print(oct10)
print(hex10)
```

✓ 0.0s

```
10
10
10
10
```

Kesirli sayılar (floating point numbers)

- Kesirli(Ondalıklı) sayının **virgüllü,** kısmı **nokta** ile belirtilir.

```
3.14
```

- İstenirse, bilimsel gösterimi belirtmek için **e** veya **E** karakteri ve sonra pozitif veya negatif bir tamsayı eklenebilir:

```
314e-2
```

✓ 0.0s

```
3.14
```

- `Kesirli(gerçel) sayı veri tipi:` **`float`**

Kesirli sayılar (floating point numbers)

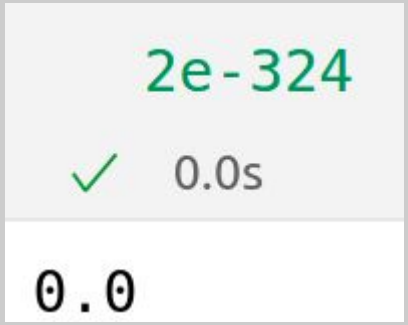
- Python kayan nokta gösteriminde IEEE-754 standartını kullanır. Bu durumda, kayan noktalı bir sayının sahip olabileceği maksimum değer yaklaşık **`1,8*10^308'`**`dir.`

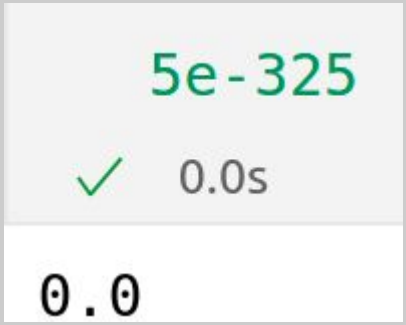
- Python bu ve daha büyük bir sayıyı **`inf`** karakter dizisi ile gösterilir;

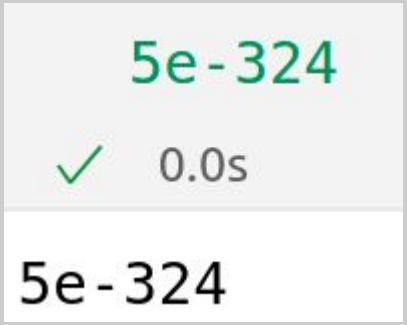
```
>>> 1.79e308
1.79e+308
>>> 1.8e308
inf
```

Kesirli sayılar (floating point numbers)

- Bir sayının sıfıra en yakın olabileceği değer yaklaşık `5,0*10^-324`'tür. Bundan sıfıra daha yakın olan her şey **sıfır** kabul edilir;







Karmaşık sayılar (complex)

- Karmaşık sayılar `<gerçel kısım>+<sanal kısım>j` şeklinde belirtilir. Örneğin:

```
>>> 2+3j
(2+3j)
>>> type(2+3j)
<class 'complex'>
```

- `Karmaşık sayı veri tipi:` **`complex`**

Karmaşık sayılar (complex)

a, b $\in$ R ve $j^2 = -1$ olmak üzere Z = a + bj biçiminde tanımlı Z sayısına karmaşık (kompleks) sayı denir. Z = a + bj karmaşık sayısında;

- a'ya Z nin gerçel (reel) kısmı '`a= Z.real`' dir.
- b'ye Z nin sanal (imajiner) kısmı '`b= Z.imag`' dir.

Bir Z karmaşık sayının gerçel (reel) kısmı: '`Z.real`', sanal (imajiner) kısmı: '`Z.imag`' komutları ile hesaplanabilir. Z sayısının gerçel (reel) kısmını elde etmek için '`Z.real`', sanal (imaginel) kısmını elde etmek için ise '`Z.imag`' komutları kullanılır.

Q: 👇Bu programın çıktısı ne olur?

```
a = 5 + 3j
b = complex(5, 3)
print ("a=",a)
print ("b=",b)
print ("a+b=",a+b) #a+b değeri
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Task: 👇Bu programın çıktısı ne olur?

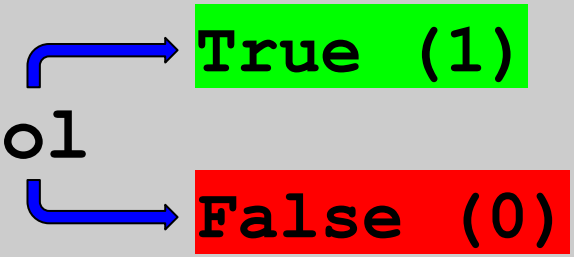
```
a = 5 + 3j
b = complex(5, 3)
print ("a=",a)
print ("b=",b)
print ("a+b=",a+b) #a+b değeri
```

✓ 0.0s

```
a= (5+3j)
b= (5+3j)
a+b= (10+6j)
```

Boolean veri tipi (bool)

- Boolean tipler, sadece iki durumu ifade eden **`True`** ve **`False`** değeri tutarlar.
- Sayısal işlemlerde ise sırasıyla **1** ve 0 tam sayısı gibi davranırlar.
- `Boolean veri tipi:` **`bool`**



bool tipinde `0/0.0` hariç tüm sayısal değerler '**True**' kabul edilir.

Q: 👇Bu programın çıktısı ne olur?

```
print (bool(3))          # ?
print (bool(-2))         # ?
print (bool(0))          # ?
print (bool(None))       # ?
print (bool(5) + bool(-3))   # ?
```



bool tipinde ‘**0**’ hariç tüm sayısal değerler ‘**True**’ kabul edilir.

Task: 👇Bu programın çıktısı ne olur?

```
print (bool(3))     # True
print (bool(-2))    # True
print (bool(0))     # False
print (bool(None))  # False
print (bool(5) + bool(-3))  # 2
#Aritmetiksel işlemlerde True:1, False:0 kabul edilir
```

✓ 0.0s

```
True
True
False
False
2
```

String veri tipi (str)

- Sözel, metinsel (text) veriler string (str) veri tipinde tutulur.
- Python dilinde ‘tek’, “çift” veya 3 `'''Tek'''` veya `"""Çift"""` tırnak arasında yazılan herşey string sabit kabul edilir.
- `String veri tipi:` **`str`**

String veri tipi (str)

- String veri tipi her türde veriyi(karakteri) içerebilir;

```
1  str_rakam = "2345"
2  str_punc = "+*?=()/%&#,..!"
3  str_text = "ali bugün bana geldi"
4  str_karma = "ali 24 yaşında mı?"
5  print(type(str_rakam))
6  print(type(str_punc))
7  print(type(str_text))
8  print(type(str_karma))
```

```
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
```

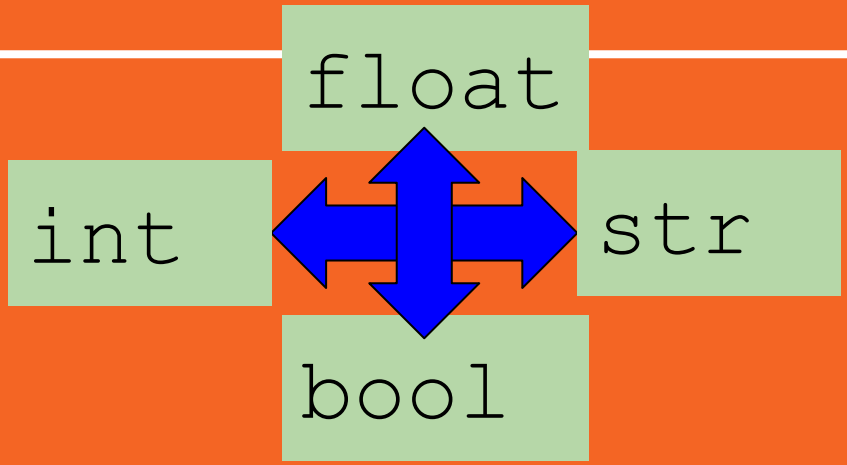
String ifadeler içerisinde **escape**/çıkış karakterleri kullanılabilir.

```
>>> print('foo\\bar')
foo\bar
```

```
ad_soy = "ali\tkoç"
print(ad_soy)
```

✓ 0.0s

```
ali	koç
```



Tip Dönüşümü (Type Conversion)

Tip Dönüşümü

Herhangi bir değerin, değişkenin veya nesnenin tipini **type()** fonksiyonu ile öğrenebiliriz;

```
my_data = "Ben bir stringim"
print(type(my_data))
```

✓ 0.0s

```
<class 'str'>
```

Tip Dönüşüm Fonksiyonları:

Tip dönüşümünde; veri, dönüştürülecek tip sarmalına alınır;

```
int():    Integer tipine dönüştürür.
str():    String tipine dönüştürür.
float():  Float tipine dönüştürür.
bool():   Boolean(True/False) tipine dönüştürür.
```

Q: 👇Bu programın çıktısı ne olur?

```
pi = 3.14
int_pi = int(pi)
flt_pi = float(int_pi)


print(int_pi)
print(type(int_pi))


print(flt_pi)
print(type(flt_pi))
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

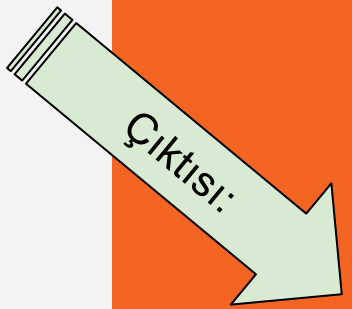
Task: 👇Bu programın çıktısı ne olur?

```
pi = 3.14


int_pi = int(pi)
flt_pi = float(int_pi)


print(int_pi)
print(type(int_pi))


print(flt_pi)
print(type(flt_pi))
```




```
3
<class 'int'>
3.0
<class 'float'>
```

Q: 👇Bu programın çıktısı ne olur?

```
int_num = 123

str_num = "456"


print(str(int_num) + str_num)

print(int_num + int(str_num))

print(int_num + str_num)
```



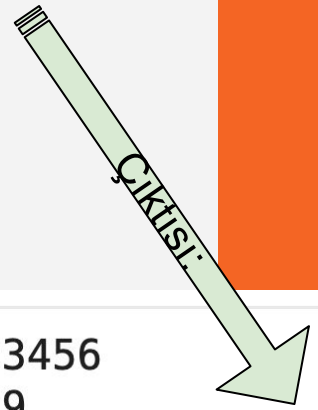
Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Task: 👇Bu programın çıktısı ne olur?

```
int_num = 123
str_num = "456"


print(str(int_num) + str_num)
print(int_num + int(str_num))
print(int_num + str_num)
```




```
123456
579
---------------------------------
TypeError
/tmp/ipykernel_1150/51623693.py in <ce
      4 print(str(int_num) + str_num)
      5 print(int_num + int(str_num))
----> 6 print(int_num + str_num)
```

OPERATÖRLER

‘`x = a + b - 5`’ işleminde ;
‘**+, -, =**’ simgelerine **operator** yani **işleç**, a, b, x, 5 gibi değişken ve değerlere ise **operand,** yani **işlenen** denir.

‘`a + b - 5`’ ise **expression / statement** yani **deyim / ifade** denir.

Aritmetiksel Operatörler

Operatör Tipi	Operatör	Örnek kullanım	operatör fonksiyon eşdeğeri
Aritmetiksel operatörler			import **operator**
Toplama	+	a + b	operator.add(a, b)
Çıkarma	-	a - b	operator.sub(a, b)
Bölme	/	a / b	operator.truediv(a, b)
Tamsayı bölme	//	a // b	operator.floordiv(a, b)
Üs alma	**	a ** b	operator.pow(a, b)
Mod (Kalanı alma)	%	a % b	operator.mod(a, b)
Çarpma	*	a * b	operator.mul(a, b)
Matris Çarpma	@	a @ b	operator.matmul(a, b)

https://docs.python.org/3/library/operator.html

mod Alma (%)

"A % B = Kalan" işlemi; "A = (B * Çarpan) + Kalan" şeklinde ifade edilebilir.

Örneğin; 9 % 4 işleminin sonucu 1'dir. Eş değeri ise `9 = (4 * 2) + 1`'dir. Yani Çarpan = 2, Kalan = 1'dir. Bu işlemde Kalan, "**mod**" olarak adlandırılır.

```
8%3
```

```
2
```

```
-8%3  # negatif sayıların yuvarlanmasında hata olabilir.
```

```
1
```

```
# bu hatayı gidermek için math.fmod() metodu kullanılır
import math
math.fmod(-8,3)
```

```
-2.0
```

Operatörlerin Önceliği

1. **Parantez içi ( )**
2. **Üs Alma (**)**
3. **Negatifleştirme (-x),** bitsel NOT (~x)
4. **Çarpma(*,@), Bölme(/, //), Mod Alma (%)**
5. Bitsel operatörler (sırası ile <<, >>, &, ^, |)
6. Karşılaştırma operatörleri (in, not in, is, is not, <, <=, >, >=, !=, ==)
7. Mantıksal operatörler (sırası ile not, and, or)
8. **Atama operatörleri (=, :=)**

Operatörlerin Önceliği

https://docs.python.org/3/reference/expressions.html

Operator	Description
`(expressions...)`, `[expressions...]`, `{key: value...}`, `{expressions...}`	Binding or parenthesized expression, list display, dictionary display, set display
`x[index]`, `x[index:index]`, `x(arguments...)`, `x.attribute`	Subscription, slicing, call, attribute reference
`await x`	Await expression
`**`	Exponentiation [5]
`+x`, `-x`, `~x`	Positive, negative, bitwise NOT
`*`, `@`, `/`, `//`, `%`	Multiplication, matrix multiplication, division, floor division, remainder [6]
`+`, `-`	Addition and subtraction
`<<`, `>>`	Shifts
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`in`, `not in`, `is`, `is not`, `<`, `<=`, `>`, `>=`, `!=`, `==`	Comparisons, including membership tests and identity tests
`not x`	Boolean NOT
`and`	Boolean AND
`or`	Boolean OR
`if – else`	Conditional expression
`lambda`	Lambda expression
`:=`	Assignment expression

Q: 👇

Aşağıdaki kod satırlarının çıktısı ne olur?



A. `(int)( 3.5 + 5.0 / 2 * 3 )`

B. `(1 + 3 ) ** (2 ** 1 * 2 / 2 / 2)`

C. `4 * 5 + 8 - (9/3)**2 + 4 % 5 // 2`

Operatörlerin Önceliği



Task: 👇

Aşağıdaki kod satırlarının çıktısı ne olur?

```
(int)( 3.5 +  5.0 / 2  * 3 )
```

```
11
```

```
(1 + 3 ) ** (2 ** 1 * 2 / 2 / 2)
```

```
4.0
```

```
(1 + 3 ** 2 ** 1 * 2 / 2 / 2)
```

```
5.5
```

Aritmetik Atama Operatörleri

Operatör	Python Kullanımı	Anlamı
+=	a+=3;	a = a+3;
-=	a-=3;	a = a-3;
*=	a*=3;	a = a*3;
/=	a/=3;	a = a/3;
//=	a//=3	a = a//3;
**=	a**=3	a = a**3

Q: 👇

Aşağıdaki kod satırlarını daha kısa nasıl yazabiliriz?

```
i = i + 5          #a
```

```
i = i + j * 5      #b
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Aritmetik Atama Operatörleri

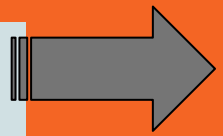
Task: 👇

Aşağıdaki kod satırlarını daha kısa nasıl yazabiliriz?

```
i = i + 5
```

```
i += 5
```

```
i = i + j * 5
```



```
i += j*5
```

Karşılaştırma Operatörleri

Operatör	Kullanımı	Fonksiyonu	Anlamı
==	a==b	eq(a, b)	a, b' ye eşit mi?
!=	a!=b	ne(a, b)	a, b' den farklı mı?
>	a>b	gt(a, b)	a, b' den büyük mü?
<	a<b	lt(a, b)	a, b' den küçük mü?
>=	a>=b	ge(a, b)	a, b' den büyük eşit mi?
<=	a<=b	le(a, b)	a, b' den küçük eşit mi?

```
import operator
operator.eq(3,3)
```

Karşılaştırma operatörleri sadece 2 değer üretir; **True** veya **False**.

Kimlik(Identity) Operatörleri

⏩ '**is**' ile '**is not**' operatörleri nesnelerin kimliğini test eder.
⏩ **x is y** ifadesi ancak x ve y aynı nesneyi işaret ediyorsa **True** sonucunu döndürür.
⏩ Bir nesnenin kimliği **id()** fonksiyonu ile saptanabilir.

Karşılaştırma Operatörleri

⏩‘**is**’ ile ‘**==**’ operatörleri aynı işleve mi sahiptir? Varsa farkları nelerdir?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Karşılaştırma Operatörleri

⏩ '**==**' operatörü içerik karşılaştırması yaparken;

⏩ '**is**' operatörü kimlik(**id**) -bellek adresi- karşılaştırması yapar.

Karşılaştırma Operatörleri

```
1  x = "bitanem"
2  y = "bitanem"
3  print(x==y)
4  print(x is y)
```

✓ 0.0s

```
True
True
```

```
1  x = "bi tanem"
2  y = "bi tanem"
3  print(x==y)
4  print(x is y)
```

✓ 0.0s

```
True
False
```

Mantıksal(Logical) Operatörler

and	Tüm ifadeleri değerlendirir ve ifadelerin tamamı **True** ise **en sondaki** ifadeyi döndürür. Aksi takdirde, **False** olarak değerlendirilen ilk ifadeyi döndürür.
or	Tüm ifadeleri (soldan sağa doğru) değerlendirir ve ifadelerin tamamı **False ise en sondaki** ifadeyi döndürür. Aksi takdirde, True olarak değerlendirilen ilk ifadeyi döndürür.
not	Tersine çevirir. Yani True ise False, False ise True döndürür.

```
print(1 and 5.5 and "ali")
```

```
ali
```

```
print(1 or 5.5 or "ali")
```

```
1
```

```
not(5)
```

```
False
```

Neler Truth Neler Falsy?

- None
- Sıfır değeri: 0, 0.0, 0j
- İçi boş string ve koleksiyonlar:

  '', [], {}, ()

  Yukarıdakiler haricindeki her şey True kabul edilir.

Q: Çıktısı nedir?

```
print(3 and "depar")
print(0 and "depar")
```

```
print(3 or "depar")
print(0 or [])
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Neler Truth Neler Falsy?

```
print(3 and "depar")
print(0 and "depar")
```

```
depar
0
```

```
print(3 or "depar")
print(0 or [])
```

```
3
[]
```

Mantıksal Operatörlerin Önceliği

1. not
2. and
3. or

```
bool_var = False or True and not True
        #                       (1)False
        #  False or True and False
        #                  (2)False
        #  False or False
        #    (3) False
print(bool_var)
```

```
False
```

any & all Operatörleri

any(*iterable*)

Herhangi bir değer true ise `True` döndürür. Eğer iterable nesne tamamen boş ise False döndürür. Örnek:

```
any(["", 0, False, 1, 0.0])
```

✓ 0.0s

```
True
```

all(*iterable*)

Tüm değerler true ise True döndürür. Iterable nesne tamamen boş ise yine True döndürür. Örnek:

```
1 all(["ali", 5, [1,2,3], -5, 9.8])
```

✓ 0.0s

```
True
```

any & all Operatörleri

```
print(any([False, False, False]))
#??

print(any([False, False, True]))
#??

print(all([False, True, True]))
#??

print(all([True, True, True]))
#??
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

any & all Operatörleri

```
print(any([False, False, False]))
False

print(any([False, False, True]))
True

print(all([False, True, True]))
False

print(all([True, True, True]))
True
```

String Formatlama (f-strings)

Dr. **Bülent Çobanoğlu**

cobanoglubulent[at]gmail.com

Hatırlama

String değişkenler; içerisinde her türlü karakterin (harflerin, sayıların ve +,-,&, gibi özel simgelerin) saklandığı karakter dizileridir;

- Karakter dizileri, birden fazla karakterin 'tırnak içerisine alınması' ile oluşur.
- Yazılım dünyasında karakter dizileri 'string' olarak adlandırılır.
- Bir stringi oluşturan karakterler değiştirilemez (immutable) özelliktedir.
- 'Tek' veya "çift tırnak", üç tırnak `'''tek'''` veya `"""çift"""` içerisine yazılan her şey stringdir.
- String (karakter dizisi) değişkenleri 'str' veri tipi ile ifade edilirler.

Farklı f-string yöntemleri

```
meyve = "Elma"
sebze = "Ispanak"
miktar = 5

print("Pazardan", miktar, "kilo", meyve, "ve", sebze, "aldım.")
print("Pazardan " + str(miktar) + " kilo " + meyve + " ve " + sebze + "
aldım.")

print("Pazardan %d kilo %s ve %s aldım." %(miktar, meyve, sebze))
print("Pazardan {} kilo {} ve {} aldım.".format(miktar, me

print(f"Pazardan {miktar} kilo {meyve} ve {sebze} aldım.")
```

Klasik stil

Java stili

C stili

Python stili

```
Pazardan 5 kilo Elma ve Ispanak aldım.
```

string.format() Metotu kullanımları

```
meyve = "Elma"
sebze = "Ispanak"
miktar = 5
print("Pazardan {} kilo {} ve {} aldım.".format(miktar, meyve, sebze))

print("Pazardan {0} kilo {1} ve {2} aldım.".format(5, "Elma", "Ispanak"))


print("Pazardan {miktar} kilo {meyve} ve {sebze} aldım.".format(miktar=5, sebze="Ispanak", meyve="Elma"))


print("Pazardan {0} kilo {meyve} ve {sebze} aldım.".format(5, sebze="Ispanak", meyve="Elma"))
```

Q: 👇

“İyilik her zaman kazandırır” yazması için { } aralıklarına ne gelmelidir?



```
test = '{} {1} {} {}'.format('İyilik', 'her', 'kazandırır.', 'zaman')
print(test)
```

Q: “İyilik her zaman kazandırır” yazması için { } aralıklarına ne gelmeli?




```
test = '{0} {1} {3} {2}'.format('İyilik', 'her', 'kazandırır.', 'zaman')
print(test)
```

✓ 0.0s

```
İyilik her zaman kazandırır.
```

String Formatında Para Değerini Virgüllerle Ayırma

Python dilinde büyük sayı değerlerini (para miktarları gibi) daha kolay göstermek için normalde alt tire '_' kullanılır;

```
>>> 10_000 + 35_000  # 45000
```

Fakat istenirse sayı değerlerinin her bir binliği virgül veya başka bir karakterle aşağıdaki gibi format() fonksiyonu (veya f-string kısaltması) kullanılarak ayrıştırılabilir:

```
bakiye = 234_530_000
print(bakiye)
virgullu_bakiye = "{:,}".format(bakiye)
print(virgullu_bakiye)
```

```
234530000
234,530,000
```

Çok satırlı f-string kullanımı:

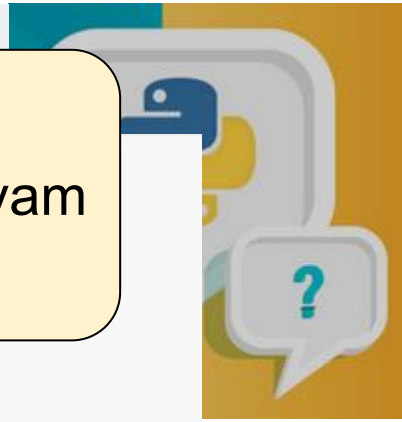


```python
name = "Bulent"
year = 30
school = "Akademi"
mesaj = f"""
    Merhaba {name},
    {year} yıldır,
    {school} de eğitim vermektedir.
    """
print(mesaj)
```

✓ 0.0s

```
Merhaba Bulent,
30 yıldır,
Akademi de eğitim vermektedir.
```

Çok satırlı f-string kullanımı:



\ ile satır bölünmeden devam edilebilir…

```
name = "Bulent"
year = 30
school = "Akademi"
mesaj = f"Merhaba {name}, " \
        f"{year} yıldır, " \
        f"{school} de eğitim vermektedir."
print(mesaj)
```

✓ 0.0s

```
Merhaba Bulent, 30 yıldır, Akademi de eğitim vermektedir.
```

Çok satırlı f-string kullanımı:

```
name = "Bulent"
year = 30
school = "Akade
mesaj = (
        f"Merhaba {name}, "
        f"{year} yıldır, "
        f"{school} de eğitim vermektedir.")
print(mesaj)
```

( ) arasına alarak satır bölünmeden devam edilebilir…



```
Merhaba Bulent, 30 yıldır, Akademi de eğitim vermektedir.
```

Built-in format() fonksiyonu



Tarihi stringe dönüştürme:

```
import datetime

tarih = datetime.datetime(2023, 5, 11, 12, 15, 58)

'{:%d/%m/%Y %H:%M:%S}'.format(tarih)

#veya
f"{tarih:%d/%m/%Y %H:%M:%S}"
```

```
'11/05/2023 12:15:58'    #çıktısı
```

Stringi sağa-sola yerleştirme



Q: Aşağıdaki programın çıktısı ne olur?

```
pi = 3.1415
genislik = 10
basamak = 4
print(f"PI sayısı..:{pi:{genislik}.{basamak}} dır.")
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Stringi sağa-sola yerleştirme



Aşağıdaki programın çıktısı ne olur?

```
pi = 3.1415
genislik = 10
basamak = 4
print(f"PI sayısı..:{pi:{genislik}.{basamak}} dır.")
```

```
PI sayısı..:      3.142 dır.
```

10 karakter

Stringi sağa-sola yerleştirme



Aşağıdaki programın çıktısı ne olur?

```
PI = 3.1415
print(f"Pi Sayısı.: {PI:.3f}")
print(f"Pi Sayısı.: {PI:10.3f}")
```

✓ 0.0s

```
Pi Sayısı.: 3.142
Pi Sayısı.:      3.142
```

10 karakter

String Metotlar : Stringi sağa-sola yerleştirme:

ljust(<), rjust(>) ve center(^) metotları

```
print(f"{'Sola yerleştim!' : <20}")
```

```
Sola yerleştim!
```

```
print('Sola yerleştim!'.ljust(20))
```

```
Sola yerleştim!
```

```
print(f"{'Sağa yerleştim!' : >20}")
```

```
    Sağa yerleştim!
```

```
print('Sağa yerleştim!'.rjust(20))
```

```
    Sağa yerleştim!
```

```
print('Merkeze yerleştim!'.center(30))
```

```
      Merkeze yerleştim!
```

```
print(f"{'Merkeze yerleştim!' : ^30}")
```

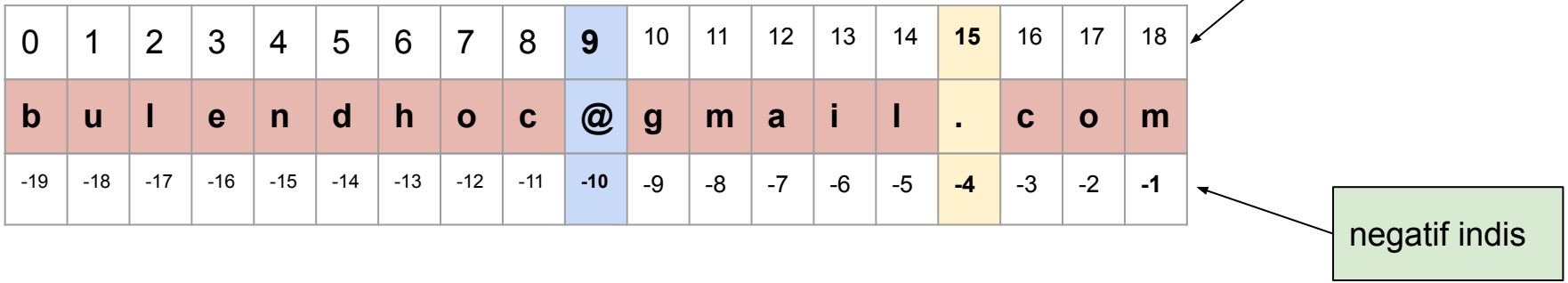
```
      Merkeze yerleştim!
```

String Metotlar: Arama işlemleri

```
# S.endswith(suffix[, start[, end]]) -> bool
# S.startswith(prefix[, start[, end]]) -> bool
url = "https://www.github.com"
print(url.startswith("http"))
print(url.endswith("com"))
print("www" in url)
```

String Metotlar: Arama işlemleri

```
email = "bulendhoc@gmail.com"
print(email.endswith(".com"))
print(email.startswith("@", 9))
print(email.startswith("@", -10))
print(email.startswith(".", -4, -1))
```



String Metotlar



Q:

Kurumsal bir firma için girilen bir e-mail adresinin doğru olup olmadığını nasıl test edebilirsiniz?

```
info@fibabank.com.tr
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

String Metotlar



Kurumsal bir firma için girilen bir e-mail adresinin doğru olup olmadığını nasıl test edebilirsiniz?

```
mail= "info@fibabank.com.tr"
print(mail.endswith("@fibabank.com.tr") and "Geçerli" or "Geçersiz")
```

String Metotlar: Arama işlemleri

```
# S.find(sub[, start[, end]]) -> int
# Aranan bulunamazsa -1 döndürür
mail = "info@fibabank.com.tr"
print(mail.find("@"))   # Bulunursa indisini/yerini verir
print(mail.rfind("@")) # Aramaya sağdan başlar
```

```
# S.index(sub[, start[, end]]) -> int
# Aranan bulunamazsa ValueError verir
mail = "info@fibabank.com.tr"
print(mail.index("@"))   # Bulunursa indisini/yerini verir
print(mail.rindex("@")) # Aramaya sağdan başlar
```

String Metotlar

Bir e-mail adresi içerisinde ‘@’ karakterinin olup olmadığını varsa pozisyonunu/konumunu veren programı kodlayalım.



```
info@fibabank.com.tr
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

String Metotlar

- **Bir e-mail adresi içerisinde '@' karakterinin olup olmadığını varsa pozisyonunu/konumunu veren programı kodlayalım.**

```
mail = "info@fibabank.com.tr"
print(mail.index("@"))   # Bulunursa indisini/yerini verir : 4
print(mail.rindex("@")) # Aramaya sağdan başlar : 4
```

String Metotlar: Değiştirme işlemleri

```
text = "Merhaba arkadaşlar"
print(text.upper())       # MERHABA ARKADAŞLAR
print(text.lower())       # merhaba arkadaşlar
print(text.casefold())    # merhaba arkadaşlar
print(text.title())       # Merhaba Arkadaşlar
print(text.capitalize())# Merhaba arkadaşlar
print(text.swapcase())    # mERHABA ARKADAŞLAR
print(text.replace("a", "o"))    # Merhobo orkodoşlor
print(text.replace("a", "o", 2))# Merhobo arkadaşlar
```

String Metotlar: Düzeltme(Edit) İşlemleri

Bir stringin başındaki ve sonundaki boşlukları veya belirli karakterleri kaldırmak için **strip** ve türevleri kullanılabilir.

```
text = "memleketlime"
print(text.strip("em"))   #'e', 'm', 'em', 'me' kombiasyonlarını siler
print(text.strip("mei"))
print(text.rstrip("me"))
```

String Metotlar: Düzeltme(Edit) İşlemleri

Bir stringin başındaki ve sonundaki boşlukları veya belirli karakterleri kaldırmak için **strip** ve türevleri kullanılabilir.

```
text = "memleketlime"
print(text.strip("em"))   #'e', 'm', 'em', 'me' kombiasyonlarını siler
print(text.strip("mei"))
print(text.rstrip("me"))
```

```
leketli
leketl
memleketli
```

String Metotlar: Düzeltme(Edit) İşlemleri



```
'TestTakeCafe'.strip('Test')
```

```
'akeCaf'
```

Sadece baştaki ‘Test’ i silmesi için ne yapabilirim?

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

String Metotlar: Düzeltme(Edit) İşlemleri

3.9 ve üzeri için geçerli

```
'TestTakeCafe'.removeprefix('Test')
```

```
'TakeCafe'
```

Sadece baştaki 'Test' i silmesi için ne yapabilirim?

String Metotlar: Düzeltme(Edit) İşlemleri

Python 3.9 ve üzeri için geçerli

```
'MiscTest'.removesuffix('Test')
```

```
'Misc'
```

```
'MiscTest'.removeprefix('Test')
```

```
'MiscTest'
```

Q: `kar = 42046198`

int değişkenini string metotlar ile ‘`420.461,98`’ formatına nasıl getirebiliriz?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Q: `kar = 420461980`

int değişkenini string metotlar ile ‘420.461,98’ formatına nasıl getirebiliriz?

```
kar = 420_461_980
s_kar = f"{kar:,}"
print(s_kar.replace(",",".",1).rstrip("0"))
```

String Metotlar: Stringi Listeye dönüştürme

⏩ **split() metodu:** s içerisinde belirtilen ayraç karakteri veya boşluklar dikkate alınarak metin ayrıştırılır ve listeye dönüştürülür

```
'1,2,3,4'.split(',')
```

```
['1', '2', '3', '4']
```

```
text = "ali topu at."
text.split()
```

```
['ali', 'topu', 'at.']
```

Bir metni kelimelerine ayrıştırma (Word Tokenization)

```
rumi = """Susmak, bilgelik meyvesinin tadına varmaktır.
Konuşmak ise, onu çürütmektir. Sevgi, seni sevmeyenler için bile yararlıdır.
Ne kadar nefret edersen et, nefretin gücüne yenilirsin.
Hoşgörü, insanın düşmanlarını bile dost edinmesine sebep olabilir."""
rumi.split()
```



```
['Susmak,', 'bilgelik', 'meyvesinin', 'tadına', 'varmaktır.', 'Konuşmak',
'ise,', 'onu', 'çürütmektir.', 'Sevgi,', 'seni', 'sevmeyenler', 'için',
'bile', 'yararlıdır.', 'Ne', 'kadar', 'nefret', 'edersen', 'et,', 'nefretin',
'gücüne', 'yenilirsin.', 'Hoşgörü,', 'insanın', 'düşmanlarını', 'bile',
'dost', 'edinmesine', 'sebep', 'olabilir.']
```

Bir metni cümlelerine ayrıştırma (Sentence Tokenization)

```
rumi = """Susmak, bilgelik meyvesinin tadına varmaktır.
Konuşmak ise, onu çürütmektir. Sevgi, seni sevmeyenler için bile yararlıdır.
Ne kadar nefret edersen et, nefretin gücüne yenilirsin.
Hoşgörü, insanın düşmanlarını bile dost edinmesine sebep olabilir."""
rumi.split('.')   # rumi.splitlines()
```

```
['Susmak, bilgelik meyvesinin tadına varmaktır',
 ' \nKonuşmak ise, onu çürütmektir',
 ' Sevgi, seni sevmeyenler için bile yararlıdır',
 ' \nNe kadar nefret edersen et, nefretin gücüne yenilirsin',
 '\nHoşgörü, insanın düşmanlarını bile dost edinmesine sebep
olabilir',
 '']
```

String Metotlar: Listeyi stringe dönüştürme

⏩ **s.join(liste)** metodu: Liste (veya başka bir veri yapısı) elemanları arasına **s** stringini yerleştirir, birleştirir.

```
liste = ['ali', 'topu', 'at.']
" ".join(liste)
```

```
'ali topu at.'
```

```
demet = ("ha", "zar")
"n".join(demet)
```

```
'hanzar'
```

String Metotlar : Bir Stringin Karakter Sayısını Öğrenme

⏩ Bir stringin karakter sayısını **len** ;
String içerisindeki bir karakterin adetini ise '**count**' komutu ile öğrenebiliriz.

Buna göre bir text'in karakter uzunluğunu ve kelime sayısını aşağıdaki gibi öğrenebiliriz;

```
text = "Herkes sana mezara kadar arkadaşlık eder"
print("Textin karakter sayısı.:", len(text))
print("Textin kelime sayısı..:", text.count(" ")+1)
```

```
Textin karakter sayısı.: 40
Textin kelime sayısı..: 6
```

String Metotlar : Stringi Ters Çevirme

⏩ **reversed():** Bir stringi ters çevirmek için join ve reversed() metotları birlikte veya dilimleme(slicing) kullanılabilir.

```
text = "alim"
print("".join(reversed(text)))
```

```
mila
```

```
text[::-1]
```

```
'mila'
```

is ile başlayan **String Metotlar**

⏩ **is** ile başlayan fonksiyonlar **True/False** değerlerinden birini döndürürler.

```
>>> 'a'.isalpha()
True
>>> '$'.isalpha()
False
>>> 'a'.islower()
True
>>> 'a'.isupper()
False
>>> '6'.isdigit()
True
>>> 'a'.isdigit()
False
>>> '$'.islower()
False
>>> ' '.isspace()
True
>>> '\n'.isspace()
True
```

```
>>>'ABCabc1'.isalnum()
True
>>> 'ABC abc1'.isalnum()
False
>>> 'Hello'.istitle()
True
>>> 'Hello'.isascii()
True
>>> 'Helloç'.isascii()
False
>>> 'Fatih34'.isidentifier()
True
>>> '34Fatih'.isidentifier()
False
>>> '341'.isdecimal()
True
>>> 'abc1'.isdecimal()
False
```

is ile başlayan **String Metotlar**

⏩ **Bir string'in keyword olup olmadığı iskeyword() ile test edilebilir.**

```
>>> from keyword import iskeyword

>>> 'hello'.isidentifier(), iskeyword('hello')
(True, False)
>>> 'def'.isidentifier(), iskeyword('def')
(True, True)
```

Karar Yapıları (Koşullu İfadeler): if

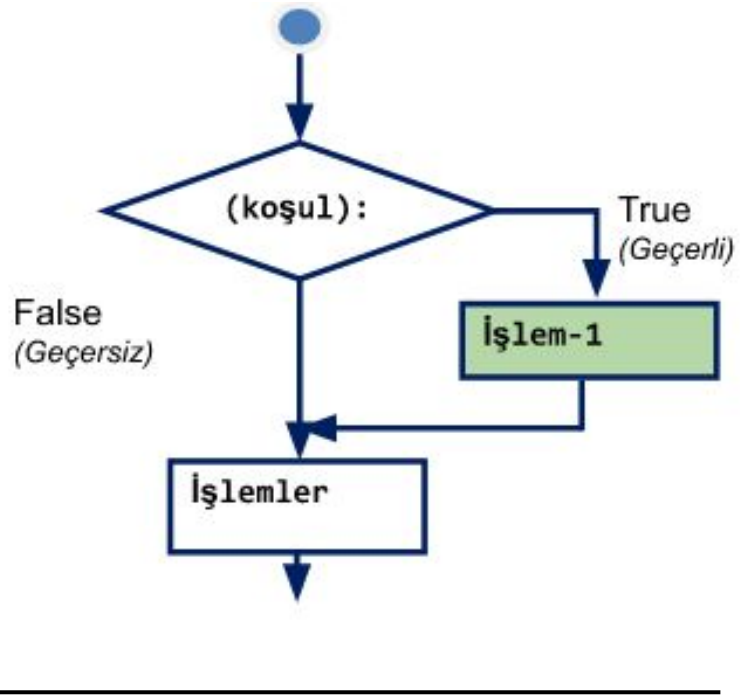
- Koşullu ifadeler oluşturmanın yolları;

  - `if`
  - `if else`
  - `elif`
  - `match-case`

if

- Tek bir koşula bağlı işlemler için;

```
...
if (koşul):
    ...
    ...
# devam....
```



- if parantezinin içerisindeki ifadede bir karşılaştırma operatörü olması gibi bir zorunluluk yoktur.



```
if (0) :        Her zaman YANLIŞ (False)
if (10):        Her zaman DOĞRU (True)
if -10:         Her zaman DOĞRU (True)
if (a):         a sıfır ise False, değilse True
if ("A"):       Her zaman DOĞRU (True)
```

if

- Girilen bir sayının sıfır mı, negatif mi veya pozitif mi olduğunu ekranda gösteren programı yazınız.

Algoritması	Python Dili Kodlaması

1. Başla
2. Gir bir sayı (S)
3. Eğer (S<0) ise
Yaz "Sayı Negatif"
4. Eğer (S>0)ise
Yaz "Sayı Pozitif"
5. Eğer (S==0)ise
Yaz "Sayı Sıfır"
6. Dur
	S = int(input('Gir bir sayı.:'))
	if S<0:
	print("Sayı Negatif")
	if S>0:
	print("Sayı Pozitif")
	if S==0:
	print("Sayı Sıfır")

Girinti (*indentation*) Meselesi



- Girilen bir sayının sıfır mı, negatif mi veya pozitif mi olduğunu ekranda gösteren programı yazınız.

Doğrusu

```
1 S = -5
2 if S<0:
3     print("Negative")
4 if S>0:
5     print("Positive")
6 if S==0:
7     print("Zero")
```

Yanlışı

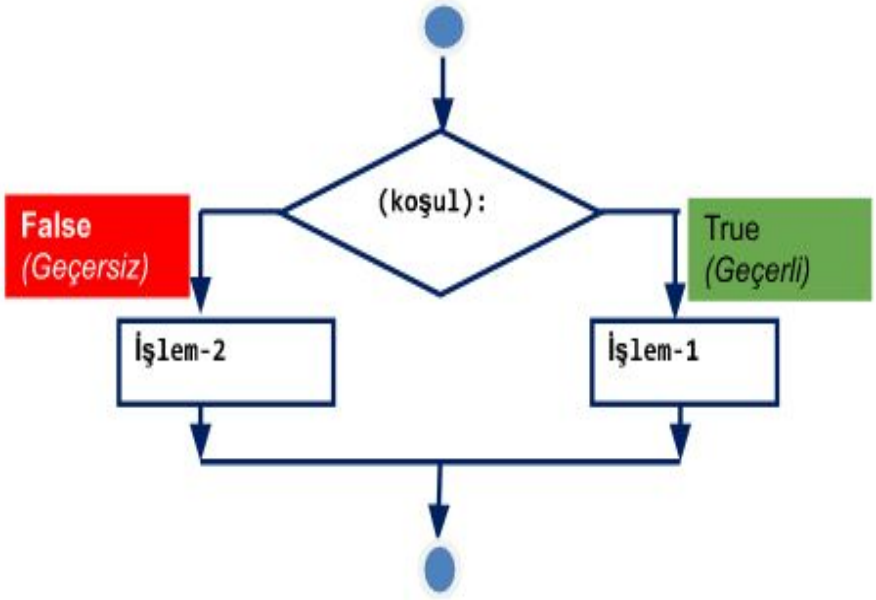
```
1 S = -5
2 if S<0:
3  print("Negative")
4 if S>0:
5      print("Positive")
6 if S==0:
7   print("Zero")
8
```

Bad Indentation (2 spaces)

if else

- Çift koşullu işlemler için;

```
if (koşul):
    işlem-1
else:
    işlem-2
```

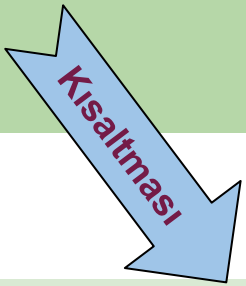



```
#veya
İşlem-1 if Koşul else işlem-2
```

if : else :

- Dışarıdan girilen Not değerine göre öğrencinin o dersten geçip, geçmediğini ekranda gösteren programı yazınız.

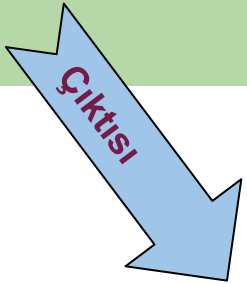
```
num = int(input ('Notu..:'))
if num >= 50:
    print ("Geçtiniz")
else:
    print ("Kaldınız")
```




```
print ("Geçtiniz" if num >= 50 else "Kaldınız")
```

Q: Verilen bir dosyanın Python dosyası olup olmadığını ekrana yazan programı kodlayınız.

```
FILE = ‘dosyam.py’
....
```




```
dosyam.py bir Python dosyasıdır
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

if : else :

- Verilen bir dosyanın Python dosyası olup olmadığını ekrana yazan programı kodlayınız.

```
FILE = "dosyam.py"
if FILE.endswith("py") or FILE.endswith("ipynb"):
    print(f"{FILE} bir Python dosyasıdır")
else:
    print(f"{FILE} bir Python dosyası değildir")
```

Çıktısı

```
dosyam.py bir Python dosyasıdır
```

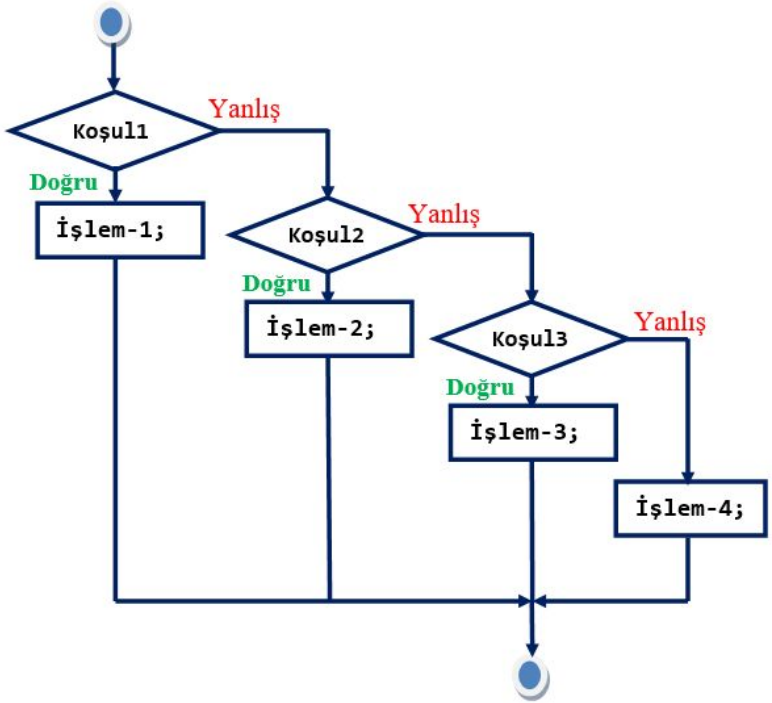
if : elif :

Üç ve daha fazla koşul durumunda **if** ve **elif** komutları merdiven şeklinde alt alta kullanılır.

elif: 'else if (Değilse Eğer)' anlamında kullanılan bir komutun bir kısaltmasıdır.

```
if (koşul1):
    işlem-1
elif (koşul2):
    işlem-2
elif (koşul3):
    işlem-3
#...

else:
    işlem-N
```



if : elif :

TCDD trenlerinde bilet fiyatında yaşa göre yapılacak indirimler yanda verilen tabloya göre hesaplanmaktadır.

Girilen yaş değerine göre indirimli bilet fiyatını gösteren programı kodlayalım.

Yaş	İndirim Oranı
0 - 6 arası	Ücretsiz
7 – 12 arası	%50
13 – 26 arası	%20
27 - 59	%0 *(indirim yok)*
60 – 64	%20
65 ve üzeri	%50

if : elif :

Yaş	İndirim Oranı
0 - 6 arası	Ücretsiz
7 – 12 arası	%50
13 – 26 arası	%20
27 - 59	%0 *(indirim yok)*
60 – 64	%20
65 ve üzeri	%50

```
Fiyat = 100
yas = int (input('Yaşınız.:'))
print ("Normal Bilet Fiyatı.:", Fiyat)
if yas>0 and yas<=6:
    iFyt = 0                    # Ücretsiz
elif yas>=7 and yas<=12:
    iFyt = Fiyat/2              # %50 indirim
elif yas>=13 and yas<=26:
    iFyt = Fiyat-Fiyat*0.20 # %20 indirim
elif yas>=27 and yas<=59:
    iFyt = Fiyat                # indirim yok
elif yas>=60 and yas<=64:
    iFyt = Fiyat-Fiyat*0.20 # %20 indirim
elif yas>=65:
    iFyt = Fiyat/2              # %50 indirim
else:
    print("Hatalı yaş değeri girildi...")

print ("İndirimli Bilet Fiyatı.:", iFyt)
```

- **Q:** (2019-Tübitak Bilgisayar Olimpiyatları Sorusu) Aşağıdaki programın ekrana “**Selin**” yazması için “**test**” değişkeni hangi aralıkta olmalıdır?



```
if test<=200:
    if test<100:
        if test<=300:
            print("Ali\n")
        else:
            print("Burak\n")
    else:
        print("Selin\n")
else:
    print("Demet\n")
```

a. 100 -300
b. 100 - 200
c. 0 - 100
d. 200 -300
e. 300 den büyük



Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

match case yapısı

Python 3.10 sürümü ile birlikte diğer dillerdeki 'switch-case' yapısına benzer ama çok daha yetenekli 'match-case' desen eşleştirme (pattern matching) yapısı dile eklenmiştir.

```
match desen:
    case <koşul_1>:  #1. eşleşme de işlem-1
            <işlem-1>
    case <koşul_2>:  #2. eşleşme de işlem-2
            <işlem-2>
    case <kosul_3>: #3. eşleşme de işlem-3
            <işlem-3>
    case _:                 # else durumu
            <işlem-4>
```

match case yapısı

Bir web sitesine girişte karşılaşılabilecek HTTP istemci hata kodlarını gösteren bir ‘http_error’ isimli fonksiyonu kodlayınız.

```
status = int(input("Web page status number..:" ))
match status:
    case 400:
        print("Bad request")
    case 404:
        print("Not found")
    case 401 | 403:  # 401 veya 403 ise
        print("Not allowed")
    case 418:
        print("I'm a teapot")
    case _:  # else
        print("Something's wrong with the internet" )
```

Döngüler (Loops)

- Hemen hemen bütün programlama dillerinde olduğu gibi Python dilinde de **iki farklı döngü yapısı** vardır. Bunlar;

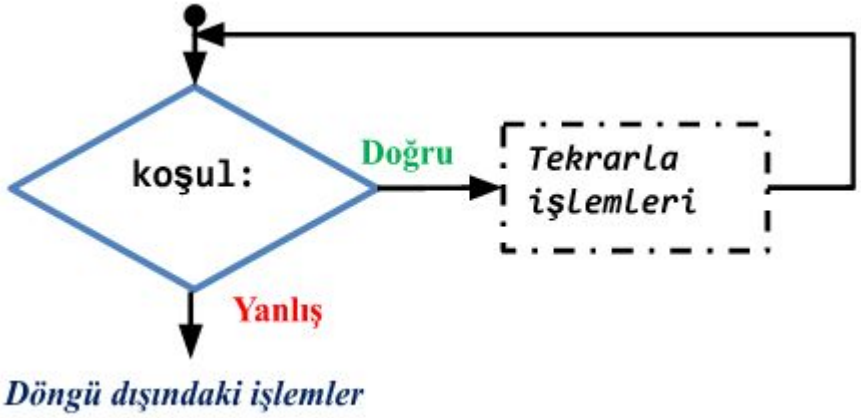
  - `while`
  - `for`

https://docs.python.org/3/tutorial/controlflow.html

while

- İşlemlerin ne kadar **tekrarlanacağının koşulun doğru** ya da **yanlış** olmasına bağlı olduğu, **koşul geçerli olduğu sürece** işlemlerin tekrarlandığı döngülerdir.

```
while (koşul):
    #işlemler
    ...
```

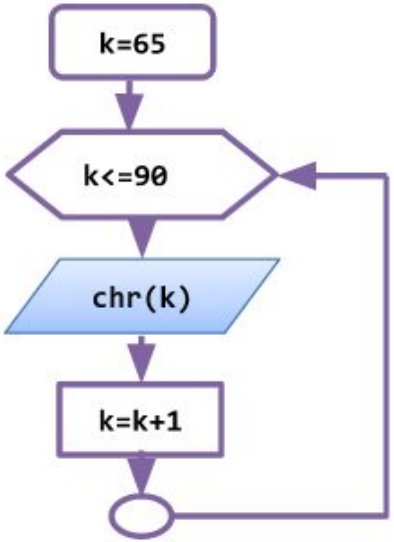


while

- A dan Z ye kadar İngilizce karakterleri ekranda gösteren programı while döngüsü ile kodlayalım. Programın örnek ekran çıktısı;

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Algoritması



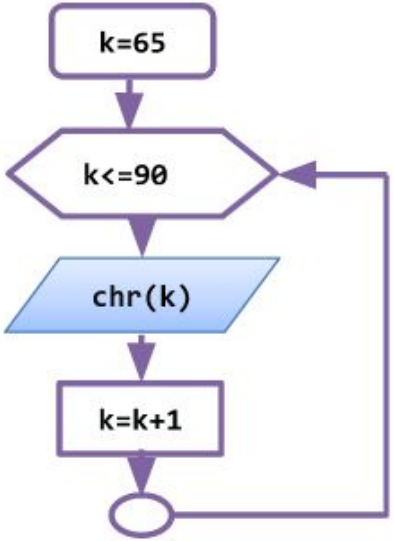
ASCII Tablosu

A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

while

- A dan Z ye kadar İngilizce karakterleri ekranda gösteren programı while döngüsü ile kodlayalım.

Algoritması




```
1   k = 65
2   while k<=90:
3       #chr(65) = 'A' demektir
4       print (chr(k), end=' ')
5       k = k + 1
```

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

while True:

- Sonsuz döngü oluşturur. Döngü içerisindekiler koşulsuz çalıştırılır. Döngüden çıkış ancak **break** komutu ile döngü kırılarak gerçekleşir

```
Farklı Sonsuz Döngü Örnekleri:
   while(1):
            ….
   while(10):
            ….
   while(-10):
            ….
```

while True:

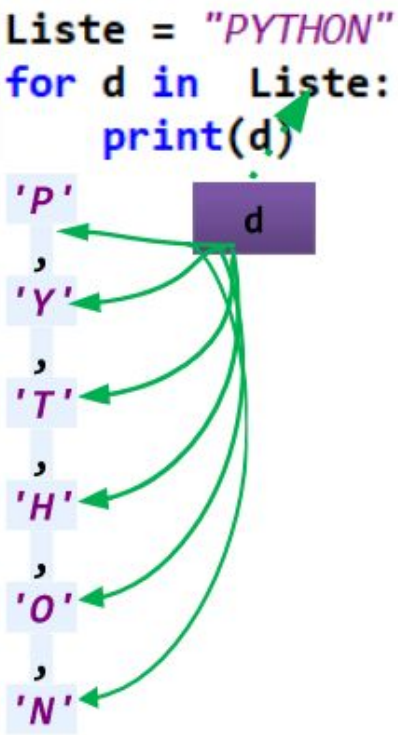
- Sonsuz döngü oluşturur. Döngü içerisindekiler koşulsuz çalıştırılır. Döngüden çıkış ancak **break** komutu ile döngü kırılarak gerçekleşir

```
Farklı Sonsuz Döngü Örnekleri:
   while(1):
            ….
   while(10):
            ….
   while(-10):
            ….
```

for

- Tekrar sayısının **baştan belli olduğu,** ardışık eleman listesinin **sırayla işleme konulduğu** döngü yapısıdır. for döngüsü **in** operatörünü kullanarak liste üzerinde ardışık (iteratif) ilerler.

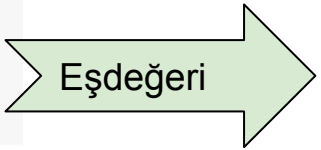
```
for d in  Liste:
    #işlem1
    #işlem2
    #....
```



for

- **range()** komutu ile ardışık bir şekilde sayılar listelendiğine göre **for döngü** yapısında doğrudan **liste** kullanılabilir mi?

```
1  for a in range(0,5):
2      print("Python")
```




```
1  for a in [0,1,2,3,4]:
2      print("Python")
```

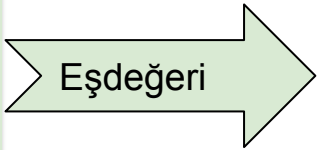
```
Python
Python
Python
Python
Python
```

```
Python
Python
Python
Python
Python
```

for - while eşdeğeri:

- 0' dan 20' e kadar ki (*20 dahil*) **çift sayıların toplamını** ekranda gösteren programı hem **for** hemde **while** döngüsü ile kodlayalım.

```
T = 0
for a in range(0,22,2):
    T = T + a
#döngü sonu
print("Toplam = ", T)
```




```
a = 0
T = 0 #Toplama değişkeni
while a < 22:
   T = T + a #Yığmalı toplama
   a = a + 2
#Döngü sonu
print ("Toplam =", T)
```

break deyimi

- Bir şarta bağlı olarak (**sonsuz döngü dah**il) herhangi bir döngüyü **sonlandırmak** için kullanılır.

```
1 for a in range(0,12,2):
2     if (a==6):
3         break
4     print(a, end=' ')
```

Console

<terminated> printKomut.py [C:\Users\bc\

```
0 2 4
```

continue deyimi

- Bir şarta bağlı olarak **döngü başı yapmak** için kullanılır.

- **break** program akışını döngü sonrası komuta,
- **continue** ise döngü başına getirir.

```
1 for a in range(0,12,2):
2     if (a==6):
3         continue
4     print(a, end=' ')
```

Console

<terminated> printKomut.py [C:\Users\bc\

0 2 4 8 10

continue deyimi

- Bir şarta bağlı olarak **döngü başı yaparak, kaldığı yerden** devam eder. Aşağıdaki programın çıktısı nedir?

```
asansor_nums = list(range(1,15))
for i in asansor_nums:
    if i==13:
        continue  # alttaki kodu çalıştırmadan döngüye devam eder.
    print(f"{i}. kattasınız")
```



Students, write your response!

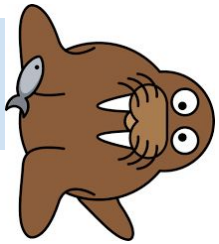
Pear Deck Interactive Slide
Do not remove this bar

Döngülerde else kullanımı

- Aynı **if-else** de olduğu gibi while/for döngülerinde de **else** kullanılabilir mi?

```
print("6 ya kaç eklersem 10 eder?")
a = 6
t = 0
while a < 10:
    a += t
    print(f"Hayır {t} eklenirse 10 olmaz")
    t += 1
else:
    print(f"Evet {t} eklenirse 10 olur")
```

Walrus operatörü (:=)

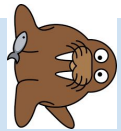


- := parantez içi bir atama operatörüdür. Python 3.8+ sonrası için geçerlidir.

```
word = input("Kelime.: ")
n = len(word)
if n >= 5:
  print(f"{word} 5 karakter veya daha büyük")
else:
  print(f"{word} 5 karakterden küçük")
```

```
word = input("Kelime.: ")
if (n := len(word)) >= 5:
    print(f"{word} 5 karakter veya daha büyük")
else:
    print(f"{word} 5 karakterden küçük")
```

Walrus operatörü (:=)

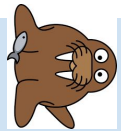


- Kullanıcı (user) şifreyi doğru girerse ekrana ‘hoşgeldiniz’ mesajını veren **değilse** sürekli şifreyi doğru girmesini bekleyen bir sorgu ekranını while döngüsü ile kodlayalım…

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Walrus operatörü (:=)



- Kullanıcı (user) şifreyi doğru girerse ATM ekranına ‘hoşgeldiniz’ mesajını veren **değilse** sürekli şifreyi doğru girmesini bekleyen bir sorgu ekranını while döngüsü ile kodlayalım…

```
while (password := input("lütfen şifrenizi girin: ")) !="python":
    pass # print("şifreyi doğru giriniz")


#döngüden çıkınca
print("şifre dogru")
```

İç içe Döngüler (Nested Loops)

- Bir döngü yapısı içerisinde başka bir döngü kullanılarak ***iç içe döngüler*** oluşturulabilir. İç içe döngülere;
  - **Zaman sistemi**;
  - **Tablo şeklindeki (satır ve sütunlardan oluşan) çarpım tablosu gibi yapılar/desenler;**
  - **Çok boyutlu diziler;**

örnek gösterilebilir.

İç içe Döngüler (Nested Loops)

- **Task** 👇 Aşağıdaki çarpım tablosunu nasıl kodlarız?

```
1 * 1 = 1       2 * 1 = 2       3 * 1 = 3       4 * 1 = 4       5 * 1 = 5       6 * 1 = 6       7 * 1 = 7       8 * 1 = 8       9 * 1 = 9       10 * 1 = 10
1 * 2 = 2       2 * 2 = 4       3 * 2 = 6       4 * 2 = 8       5 * 2 = 10      6 * 2 = 12      7 * 2 = 14      8 * 2 = 16      9 * 2 = 18      10 * 2 = 20
1 * 3 = 3       2 * 3 = 6       3 * 3 = 9       4 * 3 = 12      5 * 3 = 15      6 * 3 = 18      7 * 3 = 21      8 * 3 = 24      9 * 3 = 27      10 * 3 = 30
1 * 4 = 4       2 * 4 = 8       3 * 4 = 12      4 * 4 = 16      5 * 4 = 20      6 * 4 = 24      7 * 4 = 28      8 * 4 = 32      9 * 4 = 36      10 * 4 = 40
1 * 5 = 5       2 * 5 = 10      3 * 5 = 15      4 * 5 = 20      5 * 5 = 25      6 * 5 = 30      7 * 5 = 35      8 * 5 = 40      9 * 5 = 45      10 * 5 = 50
1 * 6 = 6       2 * 6 = 12      3 * 6 = 18      4 * 6 = 24      5 * 6 = 30      6 * 6 = 36      7 * 6 = 42      8 * 6 = 48      9 * 6 = 54      10 * 6 = 60
1 * 7 = 7       2 * 7 = 14      3 * 7 = 21      4 * 7 = 28      5 * 7 = 35      6 * 7 = 42      7 * 7 = 49      8 * 7 = 56      9 * 7 = 63      10 * 7 = 70
1 * 8 = 8       2 * 8 = 16      3 * 8 = 24      4 * 8 = 32      5 * 8 = 40      6 * 8 = 48      7 * 8 = 56      8 * 8 = 64      9 * 8 = 72      10 * 8 = 80
1 * 9 = 9       2 * 9 = 18      3 * 9 = 27      4 * 9 = 36      5 * 9 = 45      6 * 9 = 54      7 * 9 = 63      8 * 9 = 72      9 * 9 = 81      10 * 9 = 90
1 * 10 = 10     2 * 10 = 20     3 * 10 = 30     4 * 10 = 40     5 * 10 = 50     6 * 10 = 60     7 * 10 = 70     8 * 10 = 80     9 * 10 = 90     10 * 10 = 100
```

İç içe Döngüler (Nested Loops)

- **Task:** Aşağıdaki çarpım tablosunu nasıl kodlarız?

```
for A in range(1,11):
    for B in range(1,11):
        print (B,'*', A,'=',A * B, end="\t")
    print() #bir satir atla
    #B dongusu sonu
#A dongusu sonu
```

```
1 * 1 = 1	2 * 1 = 2	3 * 1 = 3	4 * 1 = 4	5 * 1 = 5	6 * 1 = 6	7 * 1 = 7	8 * 1 = 8	9 * 1 = 9	10 * 1 = 10
1 * 2 = 2	2 * 2 = 4	3 * 2 = 6	4 * 2 = 8	5 * 2 = 10	6 * 2 = 12	7 * 2 = 14	8 * 2 = 16	9 * 2 = 18	10 * 2 = 20
1 * 3 = 3	2 * 3 = 6	3 * 3 = 9	4 * 3 = 12	5 * 3 = 15	6 * 3 = 18	7 * 3 = 21	8 * 3 = 24	9 * 3 = 27	10 * 3 = 30
1 * 4 = 4	2 * 4 = 8	3 * 4 = 12	4 * 4 = 16	5 * 4 = 20	6 * 4 = 24	7 * 4 = 28	8 * 4 = 32	9 * 4 = 36	10 * 4 = 40
1 * 5 = 5	2 * 5 = 10	3 * 5 = 15	4 * 5 = 20	5 * 5 = 25	6 * 5 = 30	7 * 5 = 35	8 * 5 = 40	9 * 5 = 45	10 * 5 = 50
1 * 6 = 6	2 * 6 = 12	3 * 6 = 18	4 * 6 = 24	5 * 6 = 30	6 * 6 = 36	7 * 6 = 42	8 * 6 = 48	9 * 6 = 54	10 * 6 = 60
1 * 7 = 7	2 * 7 = 14	3 * 7 = 21	4 * 7 = 28	5 * 7 = 35	6 * 7 = 42	7 * 7 = 49	8 * 7 = 56	9 * 7 = 63	10 * 7 = 70
1 * 8 = 8	2 * 8 = 16	3 * 8 = 24	4 * 8 = 32	5 * 8 = 40	6 * 8 = 48	7 * 8 = 56	8 * 8 = 64	9 * 8 = 72	10 * 8 = 80
1 * 9 = 9	2 * 9 = 18	3 * 9 = 27	4 * 9 = 36	5 * 9 = 45	6 * 9 = 54	7 * 9 = 63	8 * 9 = 72	9 * 9 = 81	10 * 9 = 90
1 * 10 = 10	2 * 10 = 20	3 * 10 = 30	4 * 10 = 40	5 * 10 = 50	6 * 10 = 60	7 * 10 = 70	8 * 10 = 80	9 * 10 = 90	10 * 10 = 100
```

İç içe döngüler

```
calisan = ["Ali ", "Hilal ", "Pınar "]
mod = ["mutlu", "mutsuz"]
for i in calisan:   # dış döngü
    for ii in mod:  # iç döngü
        print(i + ii)
```

✓ 0.0s

```
Ali mutlu
Ali mutsuz
Hilal mutlu
Hilal mutsuz
Pınar mutlu
Pınar mutsuz
```

Q:
Aşağıdaki metindeki emojileri ve özel karakterleri nasıl temizleyebiliriz?



```
mesaj = "+Python Dersi! kolay mı yoksa zor mu? [😦|😅]"
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Q:
Aşağıdaki metindeki emojileri ve özel karakterleri nasıl temizleyebiliriz?



```
1 mesaj = "+Python Dersi! kolay mi yoksa zor mu? [😢|😅] "
2
3 for i in mesaj:
4     if i.isalnum() or i.isspace():
5         print(i,end="")
```

```
Python Dersi kolay mi yoksa zor mu
```

Bugünkü dersi anladık mı?



